第三章：运输层

1. 概述与运输层服务

- 运输层协议是在端系统中实现的，发送端从发送应用进程接收到的报文转换成运输层分组（报文段=应用报文划分为小块并给每块加上运输层首部，生成运输层报文段）
- **核心功能**：运输层为运行在不同主机上的应用**进程**提供**逻辑通信**（而网络层为主机之间提供逻辑通信）。
- **TCP连接的本质**：TCP连接是一条逻辑连接，**只在端上可看到连接，网络核心无连接**。中间路由器对TCP连接完全视而不见，它们只处理底层的IP数据报。
- **主要协议**：TCP（面向连接、可靠）和 UDP（无连接、不可靠）。
- IP是尽力而为交付服务，是不可靠服务
- TCP提供可靠数据传输和拥塞控制

2. 多路复用与多路分解

- **多路分解**：在接收端，运输层检查报文段的首部字段，将数据定向分解到正确的套接字。
- **多路复用**：在发送端，运输层从各个套接字收集数据块，加上首部生成报文段后向下传递。
    - 套接字有唯一标识符
    - 源端口号和目的端口号来表示所交付的套接字，是16比特的数。



图 3-3　运输层报文段中的源与目的端口字段

- **标识机制**：
    - **UDP**：由二元组（目的IP地址，目的端口号）标识。只要目的IP和目的端口相同，哪怕源头不同，都会被分解到同一个套接字。
    - **TCP**：由四元组（源IP，源端口，目的IP，目的端口）标识。不同客户的并发请求，服务器会根据四元组将它们分解到不同的套接字线程。

3. 无连接运输：UDP

- **特点**：不做多余的事，不建立连接，没有拥塞控制。应用层可以更精细地控制发送什么数据以及何时发送。分组首部开销小。



图 3-7　UDP 报文段结构

- **UDP检验和 (Checksum)**：用于差错检测。将所有的16比特字相加，如果最高位产生进位，**遇到任何溢出都被回卷，即加到最低位**，最后再进行反码运算。接收方将所有内容（含检验和）相加，若**正确**无误则结果全为1。这是“端到端原则”的体现。

个 16 比特的字：

0110011001100000
0101010101010101
1000111100001100

这些 16 比特字的前两个之和是：

0110011001100000
0101010101010101
1011101110110101

再将上面的和与第三个字相加，得出：

1011101110110101
1000111100001100
0100101011000010

注意到最后一次加法有溢出，它要被回卷。反码运算就是将所有的 0 换成 1，所有的 1 转换成 0。因此，该和 0100101011000010 的反码运算结果是 1011010100111101，这就变为了检验和。在接收方，全部的 4 个 16 比特字（包括检验和）加在一起。如果该分组中没有引入差错，则显然在接收方处该和将是 1111111111111111。如果这些比特之一是 0，那么我们就知道该分组中已经出现了差错。

4. 可靠数据传输原理 (rdt)

通过一系列协议版本，逐步解决底层信道不可靠的问题：

`notcorrupt` 是一个逻辑判断条件，意思是：“接收到的分组没有损坏”（即没有比特差错）。

- **rdt 1.0**：发送端和接收端仅一个状态



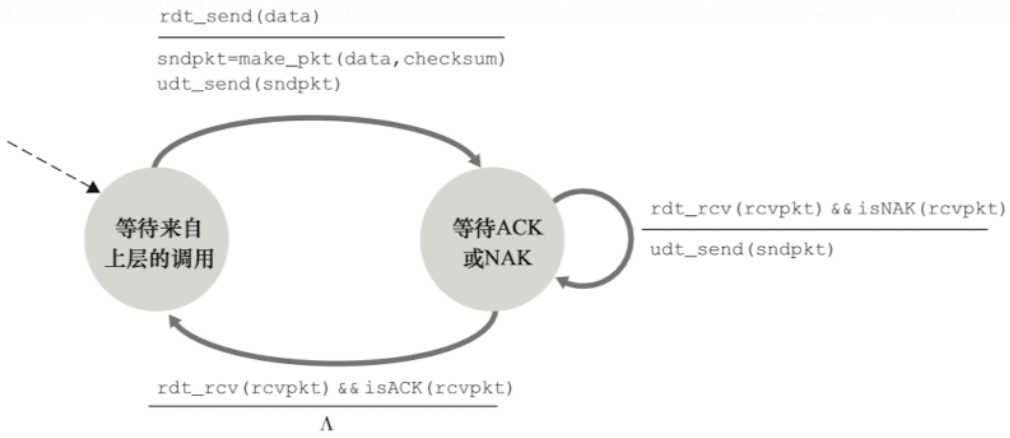
a）rdt1.0：发送端



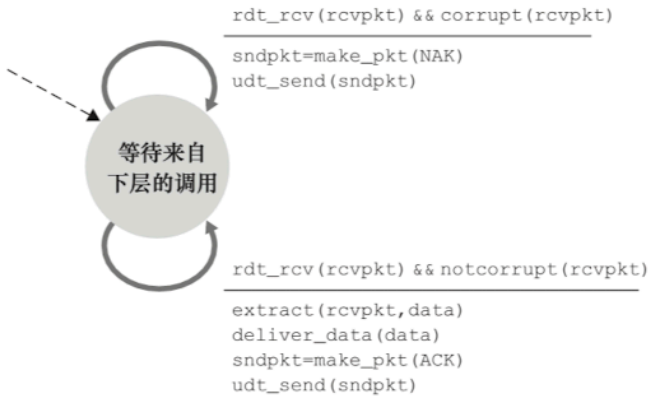
b）rdt1.0：接收端

图 3-9　rdt1.0：用于完全可靠信道的协议

- **rdt 2.0 (处理比特差错)**：使用ACK和NAK。接收方检测到分组**出错**时发送NAK，检测到分组**正确**时提取数据并发送ACK。这是一个停等协议。



a）rdt2.0: 发送端



b）rdt2.0: 接收端

图 3-10　rdt2. 0：用于具有比特差错信道的协议

- **rdt 2.1 / 2.2 (处理ACK/NAK受损)**：为防止ACK出错导致发送方盲目重传（产生冗余分组），发送方在分组中加入序号。rdt2.2是无NAK版本，通过发送带序号的冗余ACK来代替NAK。

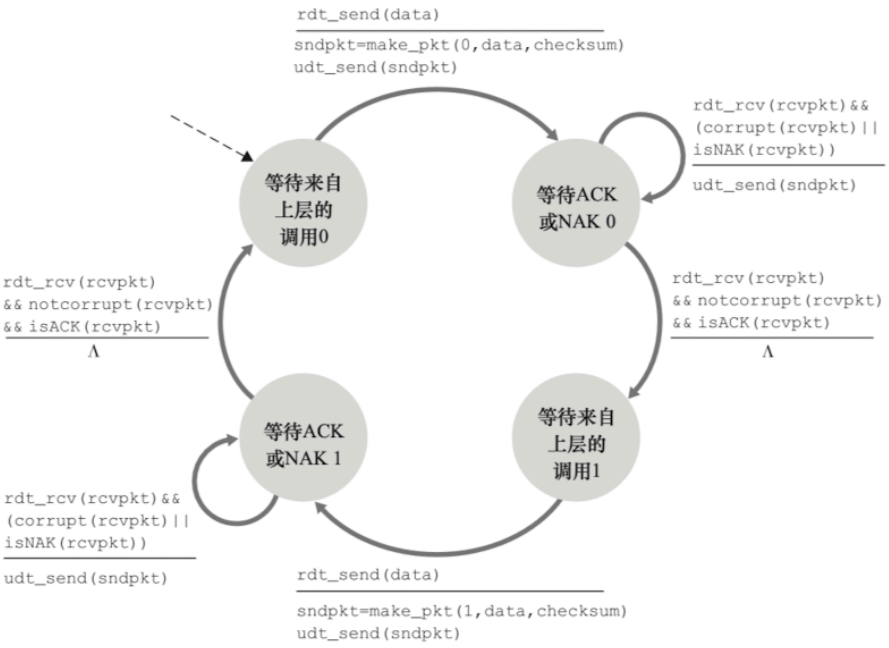


图 3-11　rdt2. 1 发送方

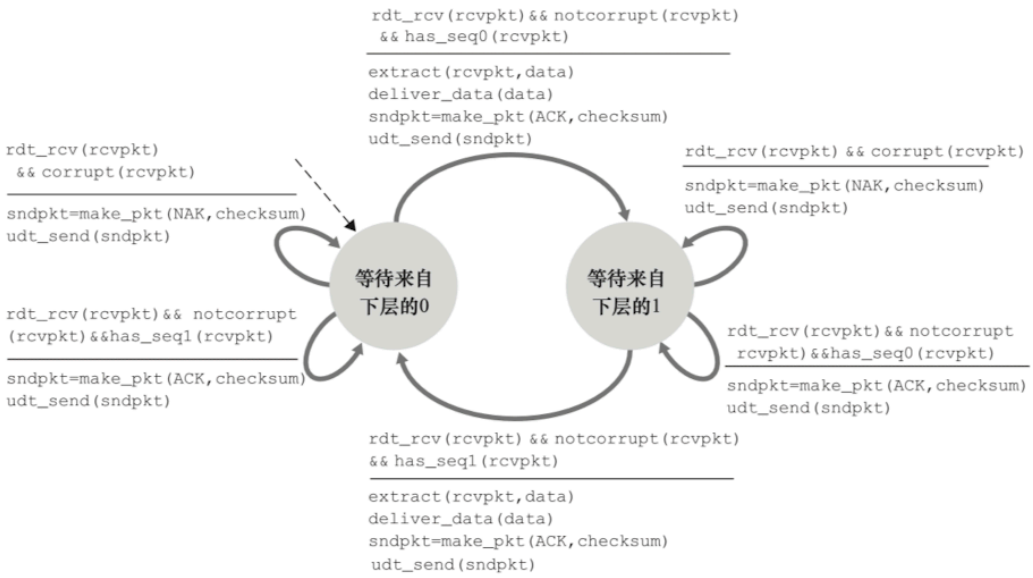


图 3-12　rdt2. 1 接收方

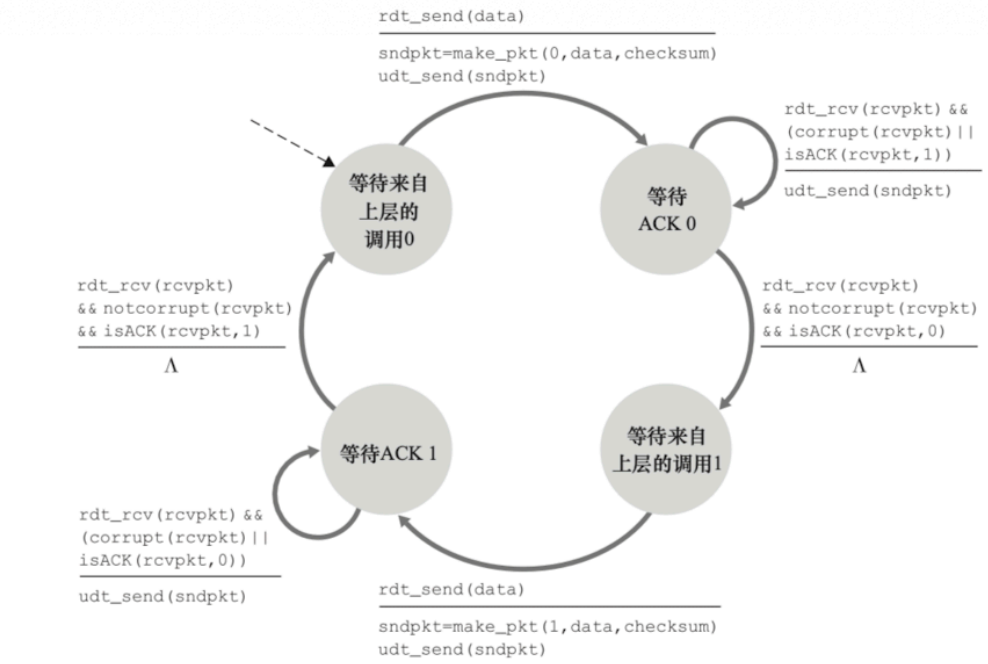


图 3-13　rdt2. 2 发送方

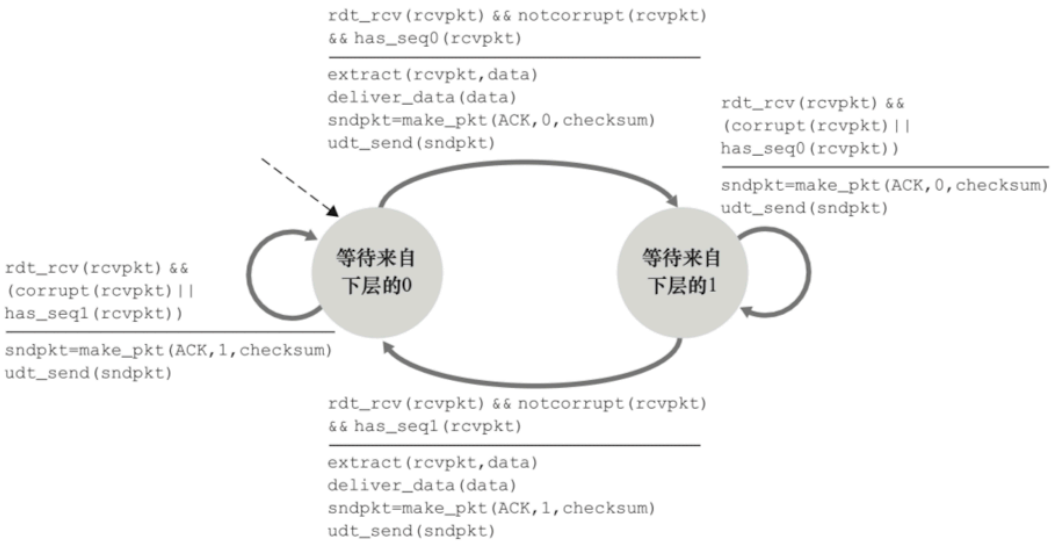


图 3-14　rdt2. 2 接收方

- **rdt 3.0 (处理丢包)**：发送方增加倒计数定时器，超时后进行重传。

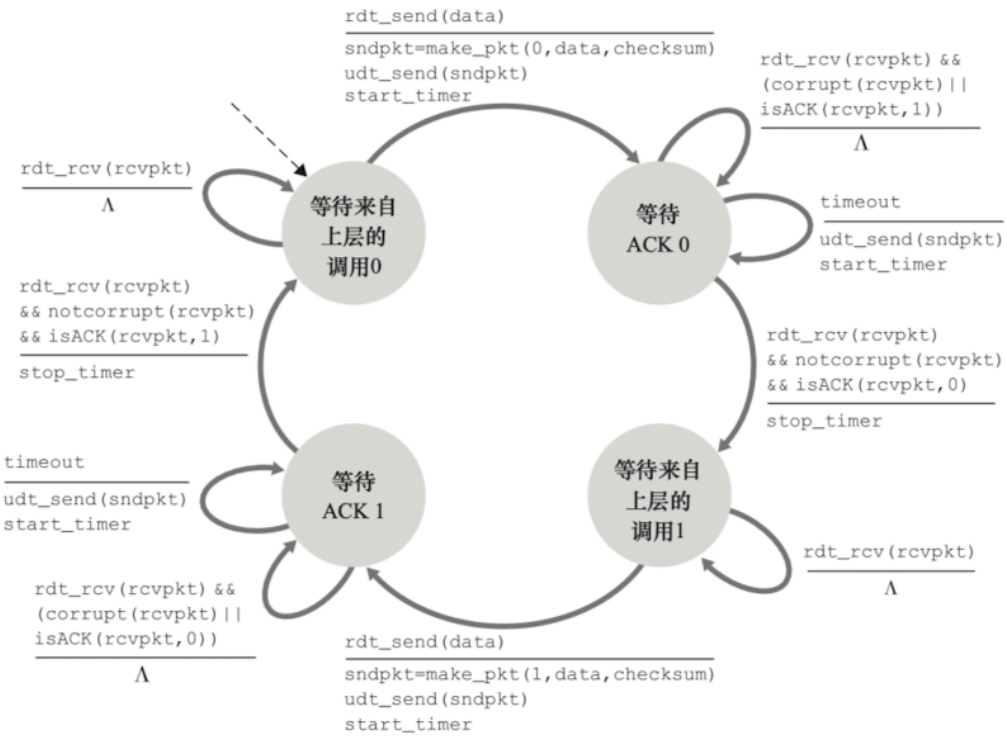


图 3-15　rdt3. 0 发送方

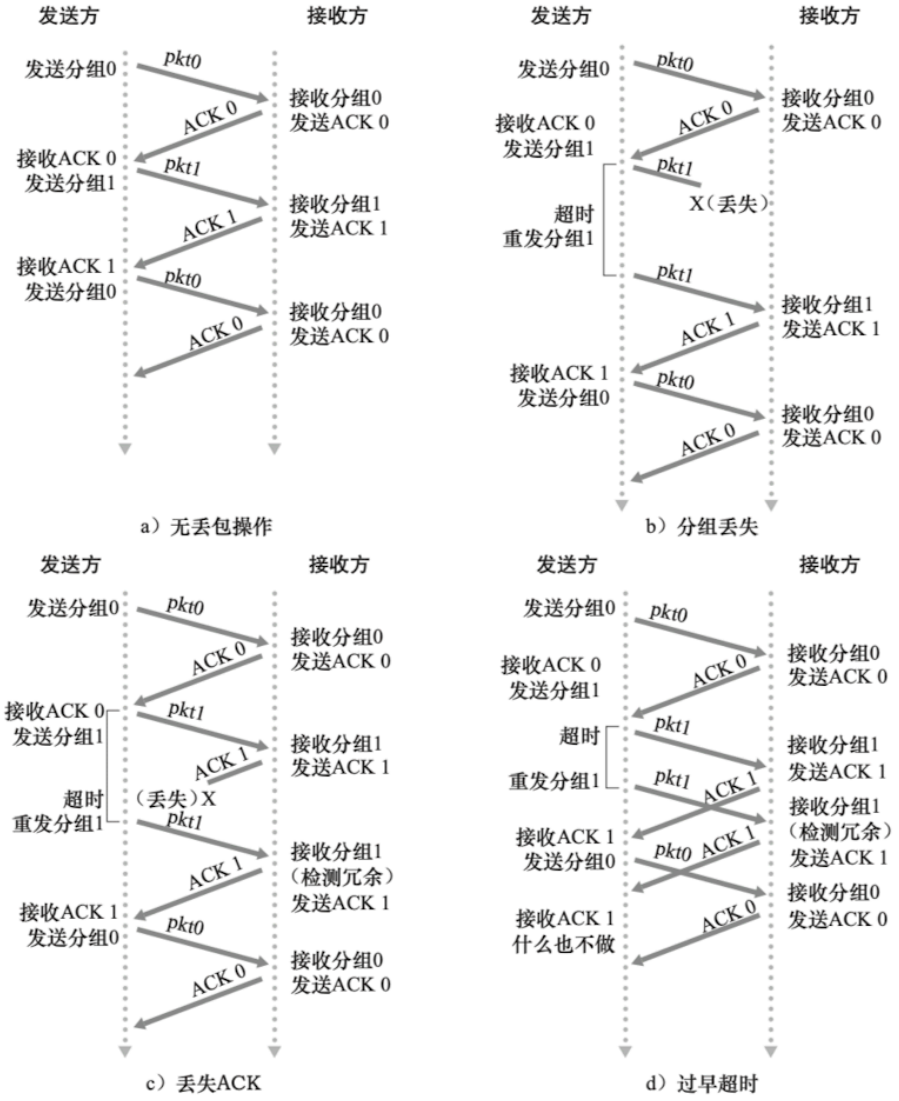
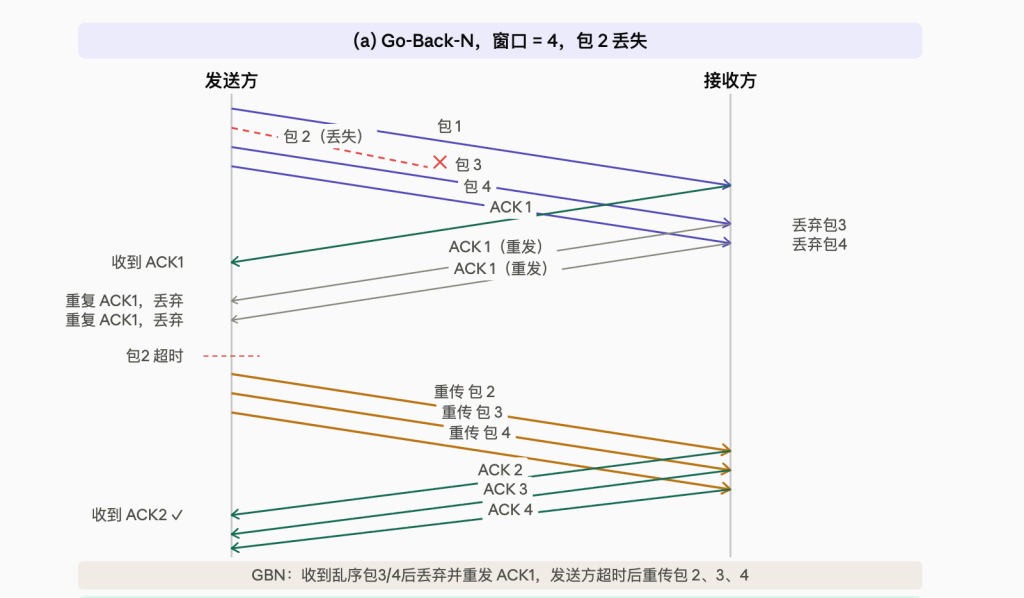
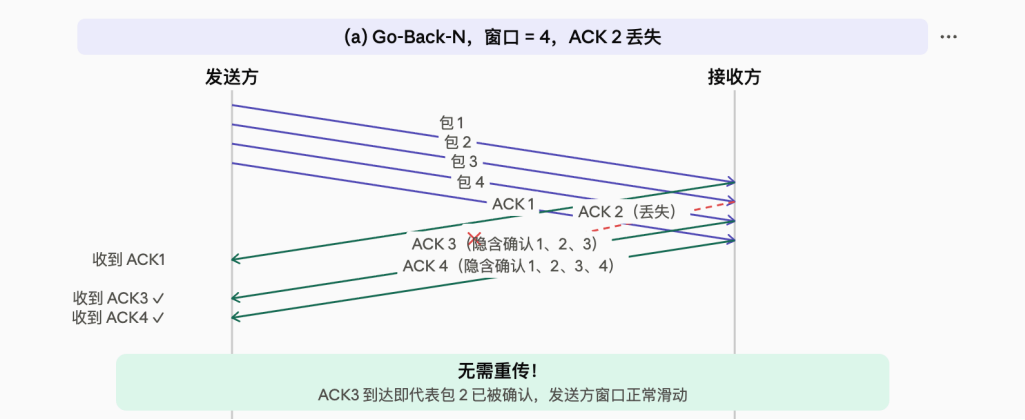


图 3-16　rdt3. 0 的运行，比特交替协议

- **流水线协议 (提高信道利用率)**：
  - **回退N步 (GBN)**：发送方可发送多个未确认分组。接收方只进行累积确认，丢弃所有失序分组。发送方超时后重传所有已发送但未确认的分组。





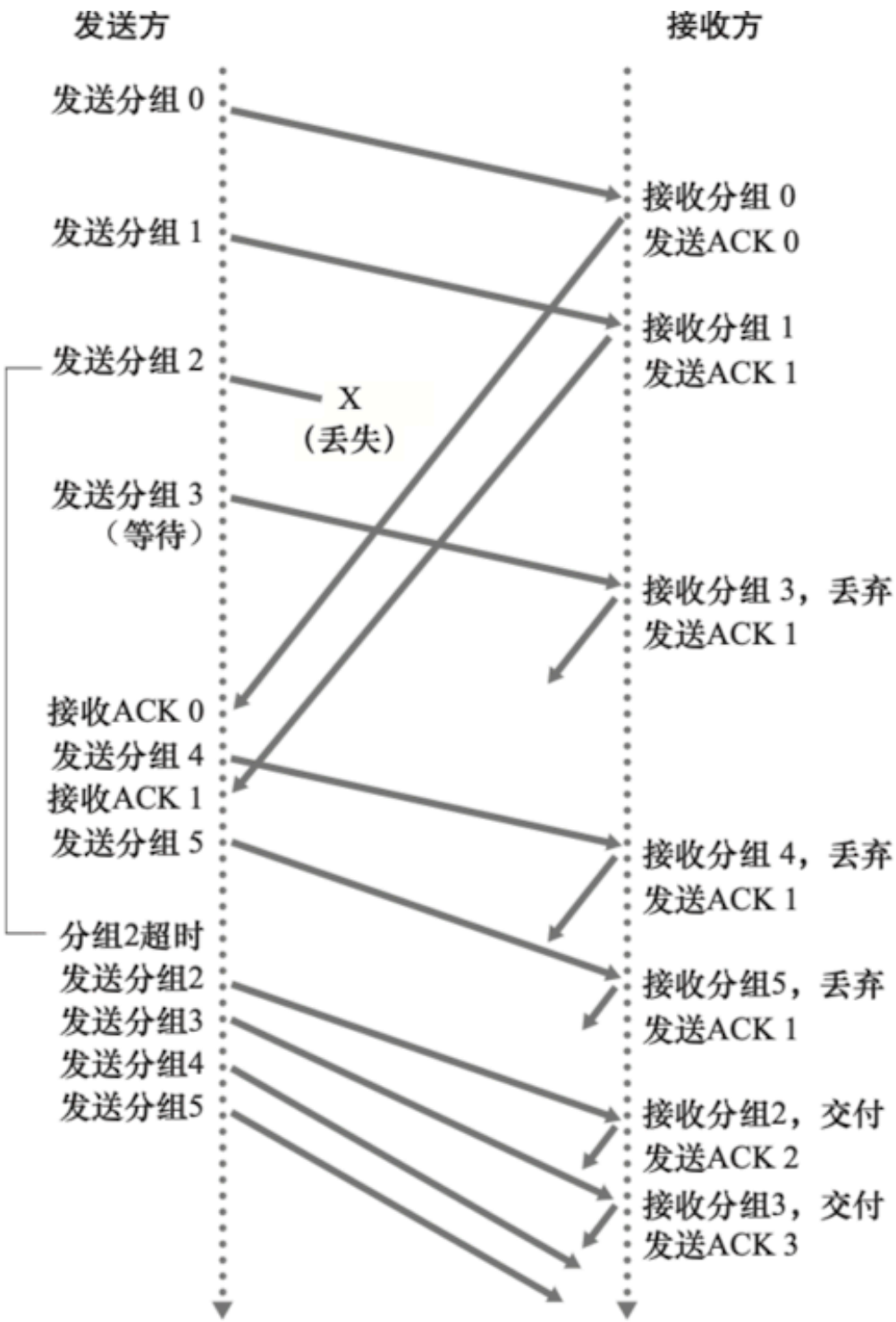
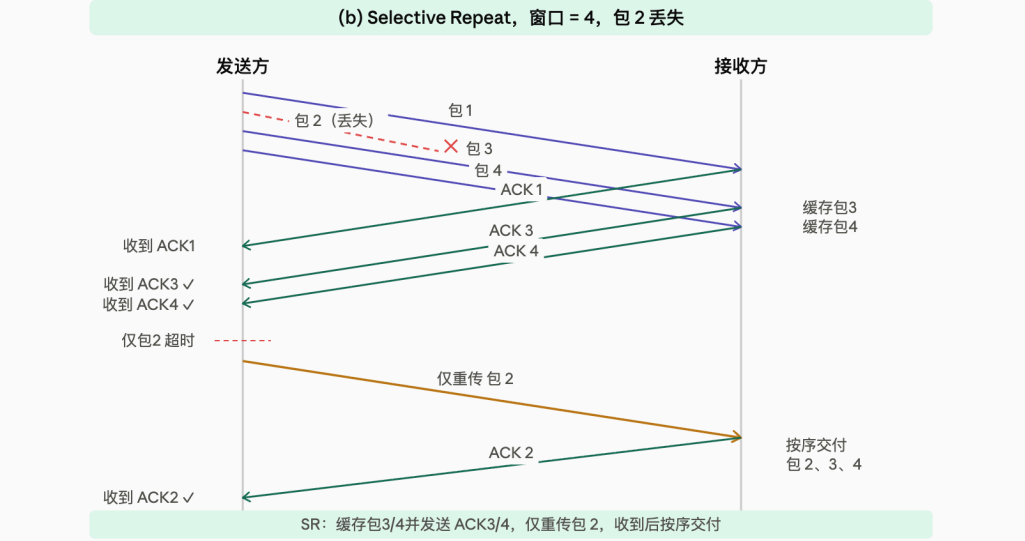
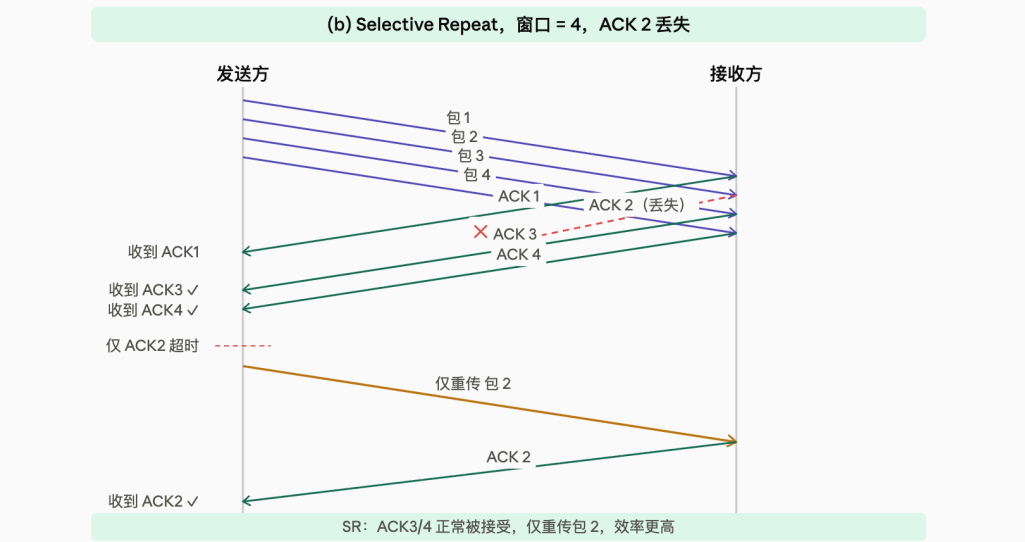


图 3-22　运行中的 GBN

- **选择重传 (SR)**：发送方仅重传疑似丢失的分组，接收方逐个确认并缓存失序分组。为了避免新旧分组序号混淆，**窗口大小必须小于或等于序号空间大小的一半**。 如果窗口过大，接收方将无法区分“新发送的分组”与“先前已接收分组的重传副本”。





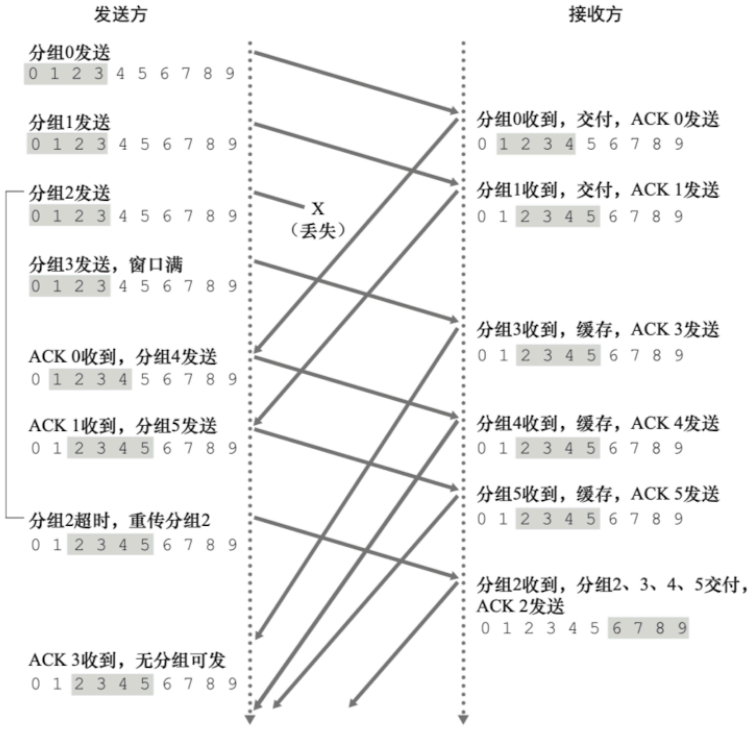


图 3-26　SR 操作

5. 面向连接的运输：TCP

- **TCP连接组成**：**TCP连接组成**包含发送缓存、接收缓存、状态变量以及套接字。
- TCP是点对点，单个发送方和单个接收方之间连接。三次握手：
  - 客户发送SYN特殊报文段
  - 服务器发送SYNACK特殊报文段回应
  - 用户发送ACK确认报文段，该报文段可以承载有效载荷
- 四次挥手：

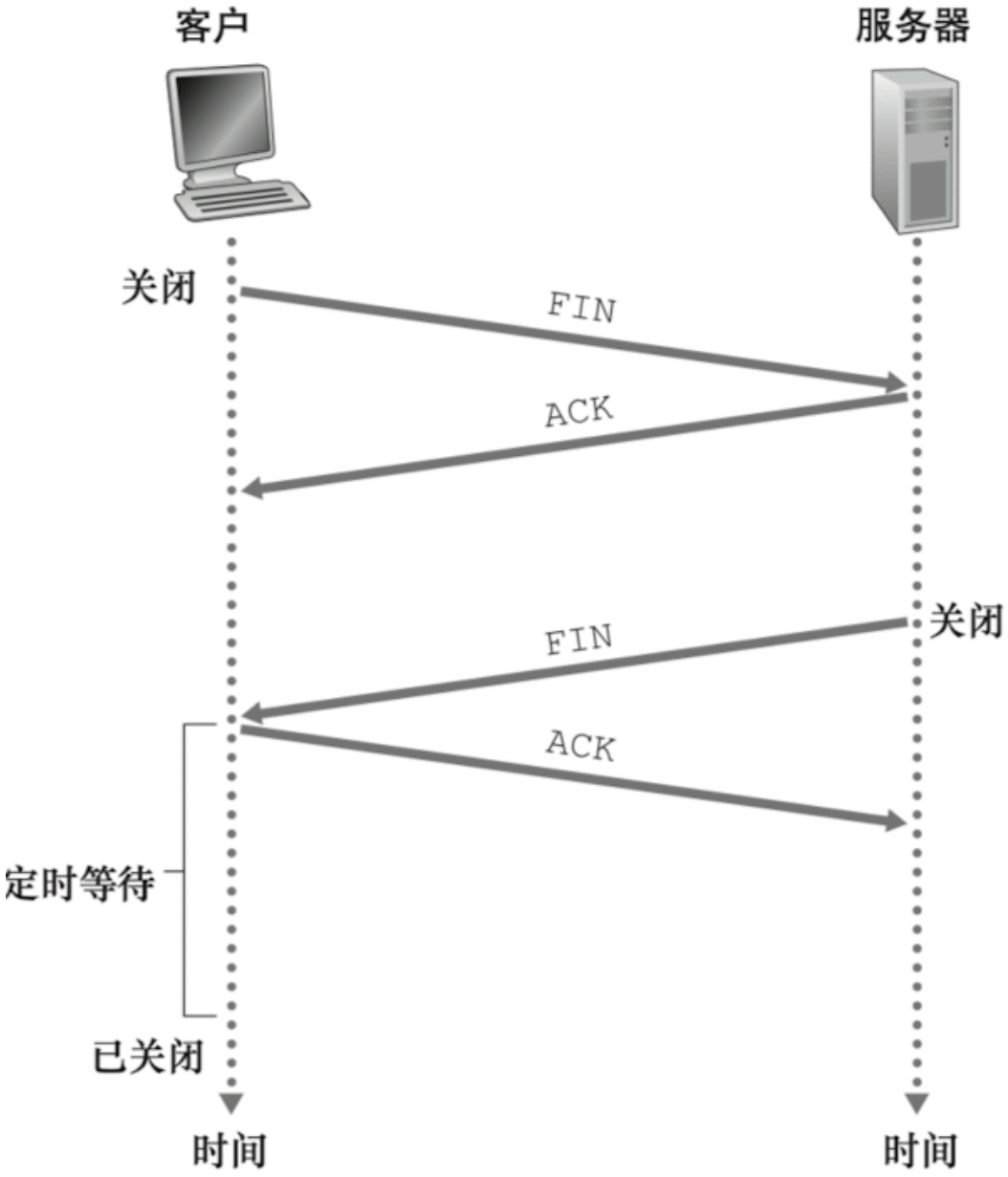


图 3-40　关闭一条 TCP 连接

- **TCP报文段结构**（最核心的两个字段）：

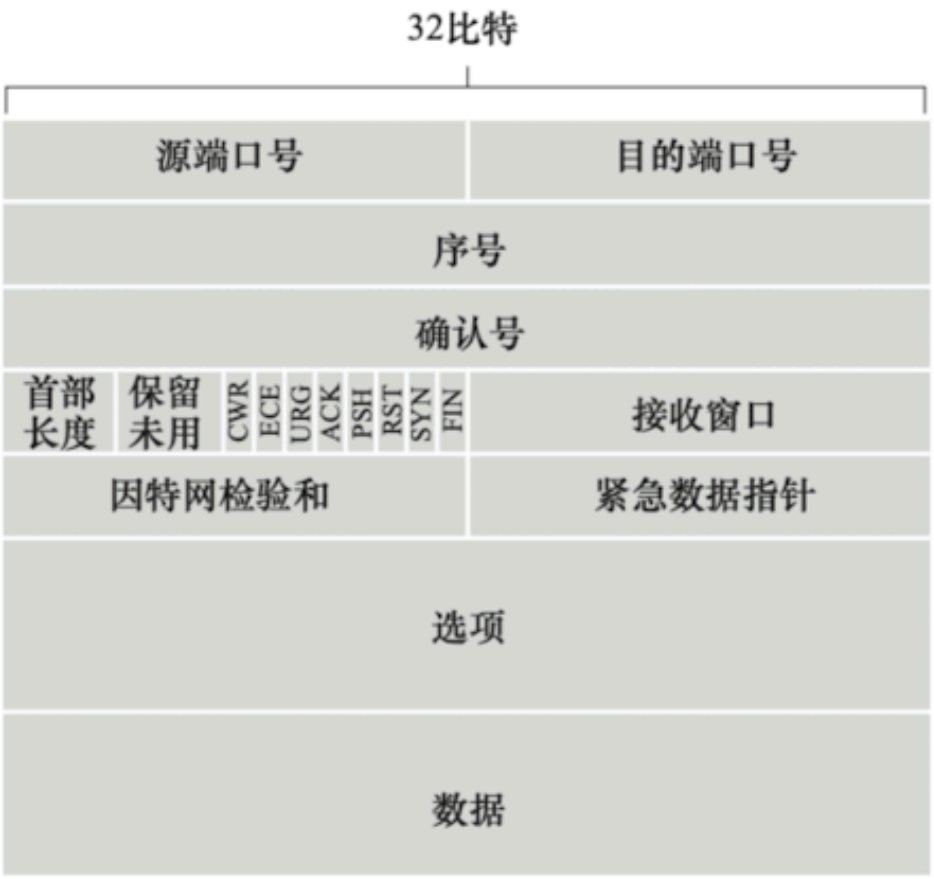


图 3-29　TCP 报文段结构

- **序号**：代表**这段第一个字节的编号**（即该报文段首字节在整个传输字节流中的编号）。
- **确认号**：代表**期望收到的下一个字节的序号**（TCP采用累积确认）。例如发送方发了**8字节数据**（序号92），接收方回送的ACK就是原序号加8，即**确认号100**。

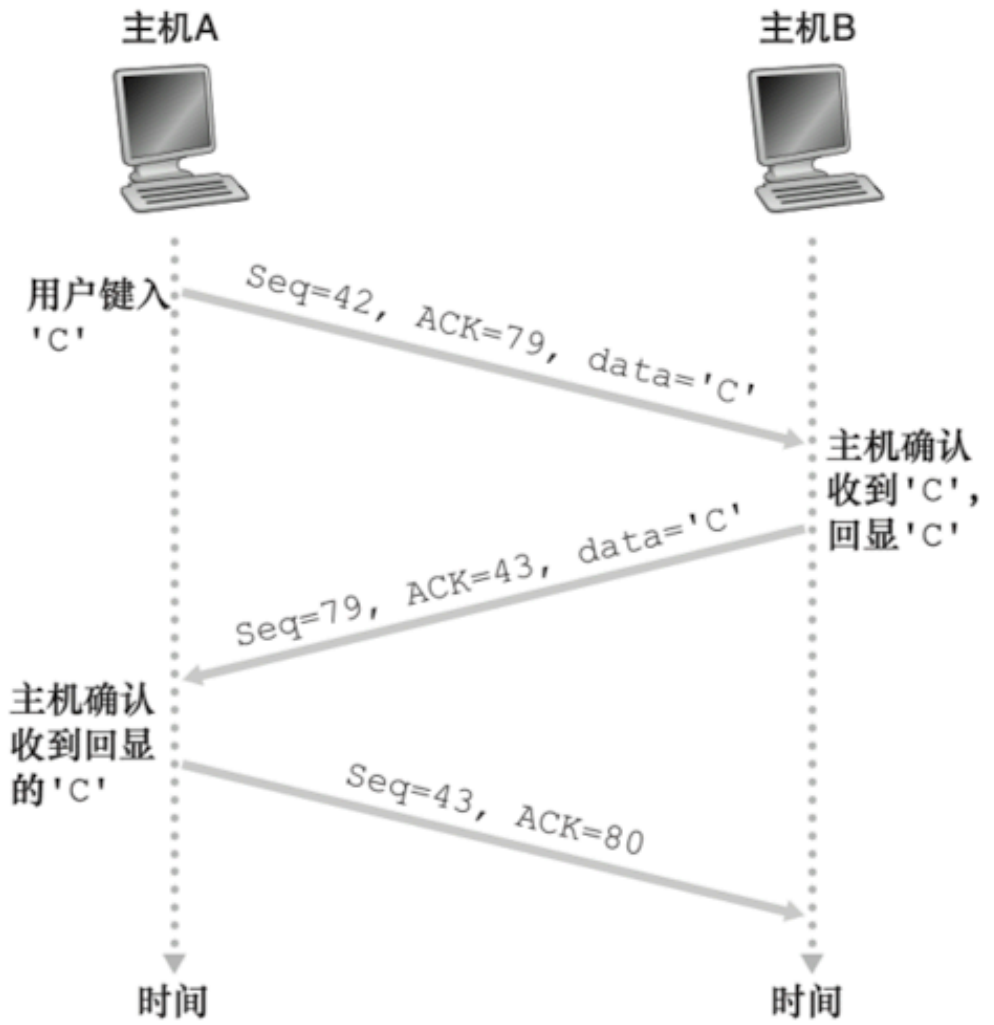


图 3-31　一个经 TCP 的简单 Telnet 应用的确认号和序号

- **超时与RTT估计**：TCP维持一个平滑的估计往返时间（EstimatedRTT）和偏差值（DevRTT）。发生超时后，超时间隔自动翻倍。

  第一个公式：RTT 估算 (EstimatedRTT)

  这个公式是为了得到一个平滑的、稳定的往返时间（RTT）预估值。

  $$EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$$

  - **怎么来的**：网络波动很大，直接用某一次的测量值（SampleRTT）会不稳定。所以 TCP 采用了一种"指数加权移动平均"的方法，把旧的预估值和新的测量值进行加权合并。
  - **变量表示**：
    - $EstimatedRTT$：**当前的预估往返时间。**
    - $SampleRTT$：**本次测量的实际**往返时间（从发送报文到收到确认的时间）。
    - α：**权值（推**荐值为 0.125），表示新数据对预估值的影响程度。

第二个公式：RTT 偏差 (DevRTT)

这个公式是为了衡量网络“抖动”得有多厉害，即波动有多大。

$$DevRTT = (1-\beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

- **怎么来的**：如果网络时延波动大，SampleRTT 离预估值 EstimatedRTT 就会很远。这个公式计算了这两者差值的平均偏差。
- **变量表示**：
  - $DevRTT$：**RTT** 的偏差预估值。
  - $|SampleRTT - EstimatedRTT|$：实际值与预估值之间的绝对偏差。
  - β：**权值**（推荐值为 0.25）。

第三个公式：重传超时设置 (TimeoutInterval)

这是最终目的：决定“到底等多久才算丢包”。

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

- **怎么来的**：如果只用 EstimatedRTT，只要稍微慢一点点就会触发没必要的重传。所以要留出一部分“余量”（Safety Margin）。DevRTT 越大，说明网络越不稳定，余量就要留得越大。
- **变量表示**：
  - $TimeoutInterval$：最终的超时等待时间。
  - $4 \cdot DevRTT$：**给偏差留出的 4 倍余量**。

- **可靠数据传输**：引入快速重传机制——当收到3个冗余ACK时，证明后续报文段已丢失，在超时之前直接快速重传该报文段。
- **流量控制**：速度匹配机制。发送方保证已发送但未确认的数据量不超过接收方通告的接收窗口（rwnd）大小，防止接收缓存溢出。

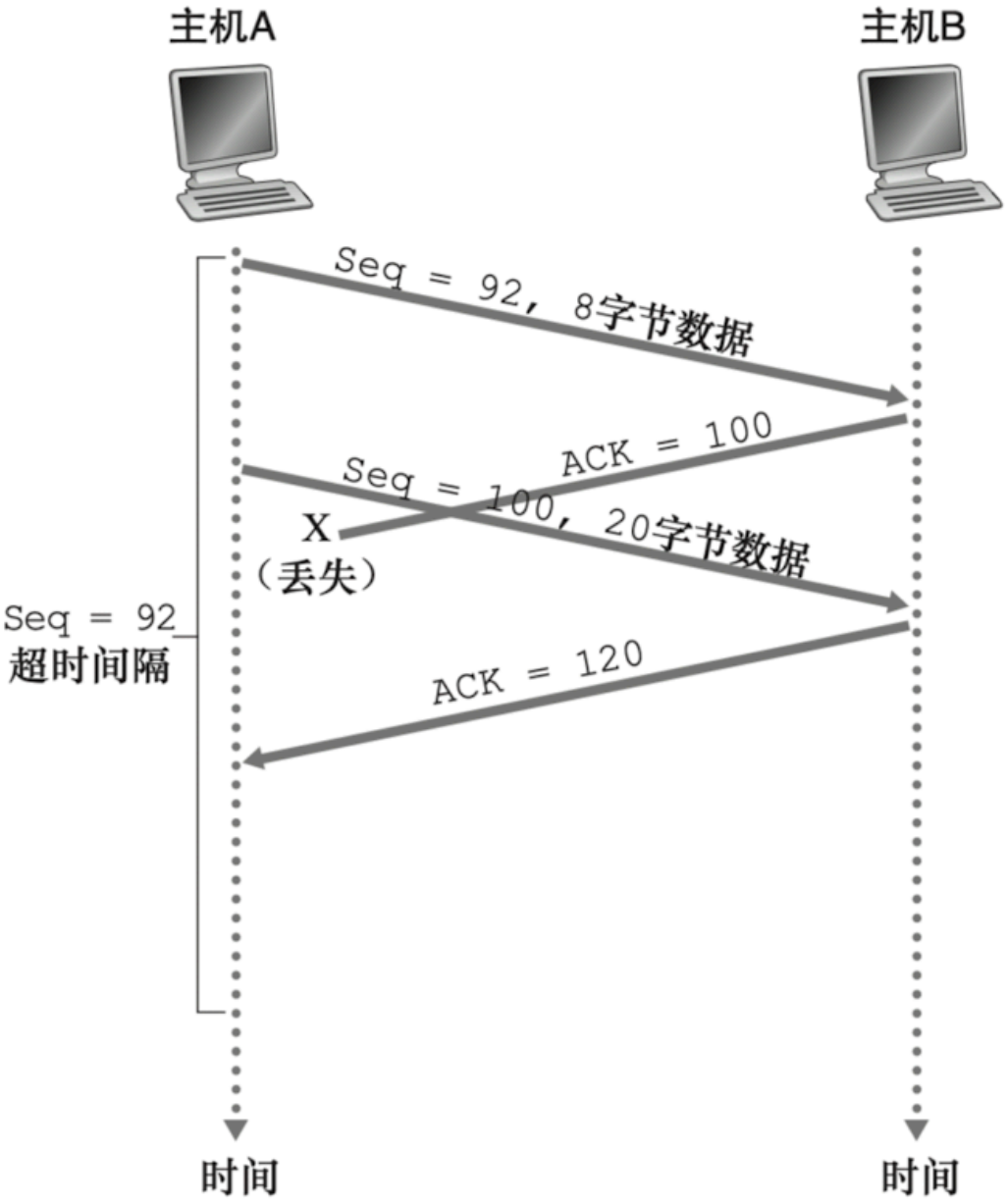


图 3-36　累积确认避免了第一个报文段的重传

6. 拥塞控制原理与代价

- **拥塞的代价**：
  - 分组在路由器中经历巨大的排队时延。
  - **重传会占用额外带宽，导致有效吞吐量下降。**
  - 当排队时延极大时，如果**发送方的超时设置不合理，会导致不必要的重传。**
  - 路由器用于转发随后被丢弃的分组的传输容量被彻底浪费了。

7. TCP 拥塞控制 (AIMD机制)

TCP拥塞控制的核心在于解决三个问题：**怎样判断发生拥塞，如何限制发送速率，用什么算法?**

- **怎样判断发生拥塞?** 发生超时，或收到3个冗余ACK。
- **如何限制发送速率?** 发送方维护一个拥塞窗口 `cwnd`，未确认的数据量不能超过 `cwnd`。

- **用什么算法?**
    1. **慢启动**：初始 `cwnd` 为1个MSS，此后每收到一个ACK增加1个MSS（每过一个RTT翻倍，呈指数增长），直到发生丢包或达到慢启动阈值 `ssthresh`。
    2. **拥塞避免**：达到阈值后，每个RTT只增加1个MSS（线性增长）。
    3. **快速恢复**：
        - TCP Tahoe（TCP的早期版本）：丢包和3个冗余ACK都把拥塞窗口设置为1MSS进入慢启动
        - TCP Reno：
            - 超时丢包：cwnd设置为1，ssthresh设置为cwnd/2
            - 检测到3个冗余ACK：快速重传并进入快速回复
                - **ssthresh = cwnd / 2** （设置新的阈值）。
                - **cwnd = ssthresh + 3 * MSS** （进入快速恢复状态，并为已离开网络的3个包留出空间）。
- **公平性**：多条TCP连接竞争同一瓶颈链路时，通过"加性增、乘性减（AIMD）"的锯齿状波动，最终会平分链路带宽。

第三章课后复习思考题 (重点整理与简明答案)

R1. 网络层和运输层的区别是什么? 答：网络层为主机之间提供逻辑通信；运输层为运行在不同主机上的应用进程之间提供逻辑通信。

R2. 什么是多路复用与多路分解? 答：多路分解是接收端运输层将数据定向到正确的套接字；多路复用是发送端运输层从各套接字收集数据封装上首部后向下传递。

R3. 考虑一个UDP应用，怎样标识目的套接字? TCP呢? 答：UDP由二元组（目的IP，目的端口）标识。TCP由四元组（源IP，源端口，目的IP，目的端口）标识。

R4. UDP相对于TCP的优势有哪些? 答：无需建立连接（无握手时延）、不维护连接状态（消耗资源少）、首部开销小（仅8字节）、发送速率不受拥塞控制限制（可精细控制）。

R5. rdt 2.0中为什么要引入序号? 答：为了解决ACK/NAK受损时，发送方盲目重传导致接收方收到冗余（重复）分组的问题。接收方通过检查序号即可判断到达的是新分组还是重传的分组。

R6. GBN和SR协议的核心区别是什么? 答：GBN采用累积确认，只维护一个定时器，出错时回退重传所有未确认分组，接收方丢弃失序分组。SR采用逐个确认，为每个分组独立维护定时器，仅重传出错的分组，接收方会缓存失序分组。

R7. 什么是TCP快速重传? 答：当TCP发送方连续收到3个对同一序列号的冗余ACK时，它就判定该报文段之后的那个报文段已经丢失，不等待定时器超时，立即重传该丢失的报文段。

R8. 流量控制和拥塞控制的区别? 答：流量控制是为了防止发送方发送太快导致**接收方缓存溢出**；拥塞控制是为了防止发送方发送太快导致**网络核心（路由器）缓存溢出**。

R9. 是非判断：在连接的整个过程中，TCP的rwnd的长度绝不会变化。 答：**错**。rwnd是动态变化的，它反映的是接收方接收缓存的当前剩余可用空间，受应用程序读取数据的速度影响。

R10. 是非判断：TCP报文段在它的首部中有一个rwnd字段。 答：**对**。TCP首部中的"接收窗口"字段就是rwnd，专门用于流量控制。

R11. 假设主机A向B发送两个紧接着的报文段，第一个序号为90，第二个序号为110。第一个报文段中有多少数据? 答：110 - 90 = **20字节**的数据。（因为序号是该段第一个字节在数据流中的编号）。

R12. 承上题，假设第一个报文段丢失而第二个到达主机B，主机B发给A的确认号是多少? 答：确认号仍然是 **90**。因为TCP采用累积确认，由于缺失了第一个报文段，主机B仍然在期望收到序号为90的字节。